Burn, little candle, burn!

Speaker notes

planned	skippable	slides
39:50	**4:50**	**46 + 3**

Each page: slide, planned time, the clock time at the end of the slide, short cues, then the full notes. **SKIP** marks slides to drop when running late. **BACKUP** slides come after the end, for questions only.

#	slide	time	at	
1	Burn, little candle, burn!	0:30	0:30	
2	Two ends of one scale	1:00	1:30	
3	Ludwig Weinzierl	0:40	2:10	
4	What does it take to do ML in Rust?	0:40	2:50	
5	01 Two frameworks	0:10	3:00	
6	What a framework does for you	1:00	4:00	
7	Candle	1:00	5:00	
8	Burn	1:00	6:00	
9	Burn 0.22 moved the backend	1:10	7:10	
10	02 Tensors	0:10	7:20	
11	One matrix product: Candle	0:50	8:10	
12	One matrix product: Burn	0:50	9:00	
13	Wrong shapes: Candle returns an error	0:45	9:45	
14	Wrong shapes: Burn panics	0:50	10:35	
15	Wrong rank: Burn does not compile	0:50	11:25	
16	Where the information lives	1:10	12:35	**skip**
17	03 Candle, batteries included	0:10	12:45	
18	A model is a download away	1:00	13:45	
19	An image: ResNet-18	1:15	15:00	
20	A sentence: MiniLM	1:20	16:20	
21	Five sentences about Bologna	1:05	17:25	
22	A chat: SmolLM2, one token at a time	1:15	18:40	
23	What it answers	0:50	19:30	
24	What Candle gives you, what you write	0:50	20:20	**skip**
25	04 Learning sin(x)	0:10	20:30	
26	Learning, in four lines	1:00	21:30	
27	Experiment 1: learn sin(x)	0:50	22:20	
28	The model, twice	0:50	23:10	
29	Training loop: Candle	0:50	24:00	
30	Training loop: Burn	0:55	24:55	
31	Same start, same data, same curve	1:10	26:05	
32	The detail that made it work	0:50	26:55	**skip**
33	05 Burn without std	0:10	27:05	
34	Four steps, one no_std crate each	1:00	28:05	
35	1 · Inference: the weights stay in flash	1:15	29:20	
36	2 · Training on the device	1:15	30:35	
37	Two things std would have given	1:00	31:35	**skip**
38	3 · Audio: teach it words	1:30	33:05	
39	4 · Someone else's model, straight from flash	1:30	34:35	
40	What it took	1:00	35:35	**skip**
41	Try it now, on your phone	1:00	36:35	
42	06 Choosing	0:10	36:45	
43	Which one?	1:30	38:15	
44	Three things I take home	0:50	39:05	
45	Try it	0:30	39:35	
46	Grazie mille	0:15	39:50	
47	Backup: three ways a model comes in			**backup**
48	Backup: quantization			**backup**
49	Backup: timing GPU code			**backup**

Burn, little candle, burn!

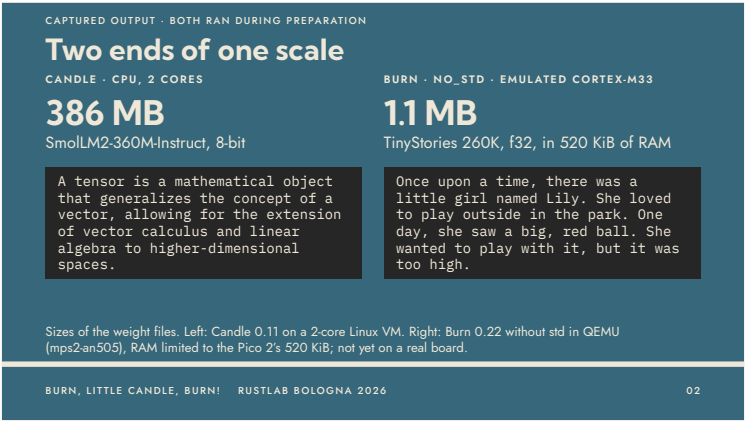


- **Greet, name, title. Smile.**
- **Promise: real code, real output, three questions.**

Good morning. I'm Ludwig. This talk is about the two machine learning frameworks you meet first in Rust: Candle and Burn.

Everything you will see today is code that ran while I prepared this talk. The code on the slides is cut from the real files, and every output is captured, not typed. Where something only compiled, or ran in an emulator, the slide says so.

Two ends of one scale



- **Left: Candle, 360M parameters, 386 MB, 2 CPU cores.**
- **Right: Burn without std, 260K parameters, 1.1 MB of f32, emulated Cortex-M33, RAM cut to 520 KiB.**
- **This talk: everything in between.**

Two outputs to start with. On the left, a chat model with 360 million parameters, 386 megabytes, running in Candle on two CPU cores. I asked it what a tensor is.

On the right, a language model with 260 thousand parameters, one megabyte of f32 weights, running in Burn without the standard library. It ran in QEMU, on an emulated Cortex-M33, the processor core of a Raspberry Pi Pico 2, and I limited its RAM to the Pico 2's 520 KiB.

Same language, same idea, three orders of magnitude apart. This talk is about what sits between these two ends.

> **Q: Did the right side run on a real Pico 2?**
> A: No. It ran in QEMU, on an emulated Cortex-M33 with the same memory limit. The firmware for the real board builds; I have not run it on hardware.

Ludwig Weinzierl



- **Rust since 2015, Rust Munich Meetup.**
- **Not an ML researcher: a Rust developer who needed ML.**
- **Candle: MiniLM embeddings for a comment score predictor.**

A few words about me. I have been writing Rust since 2015 and I co-organize the Rust Munich Meetup. My background is embedded systems, LoRaWAN and, as a hobby, retro computing.

I am not an ML researcher. I came to this as a Rust developer who wanted ML in his own programs. The first time I used Candle was for sentence embeddings: I computed MiniLM embeddings for Hacker News comments and trained a small scoring model on top, with XGBoost, to predict comment scores. For that narrow task the specialized model was easy to make better than just asking an LLM. That is the kind of practical ML this talk is about.

You will meet MiniLM again in a few minutes.

What does it take to do ML in Rust?



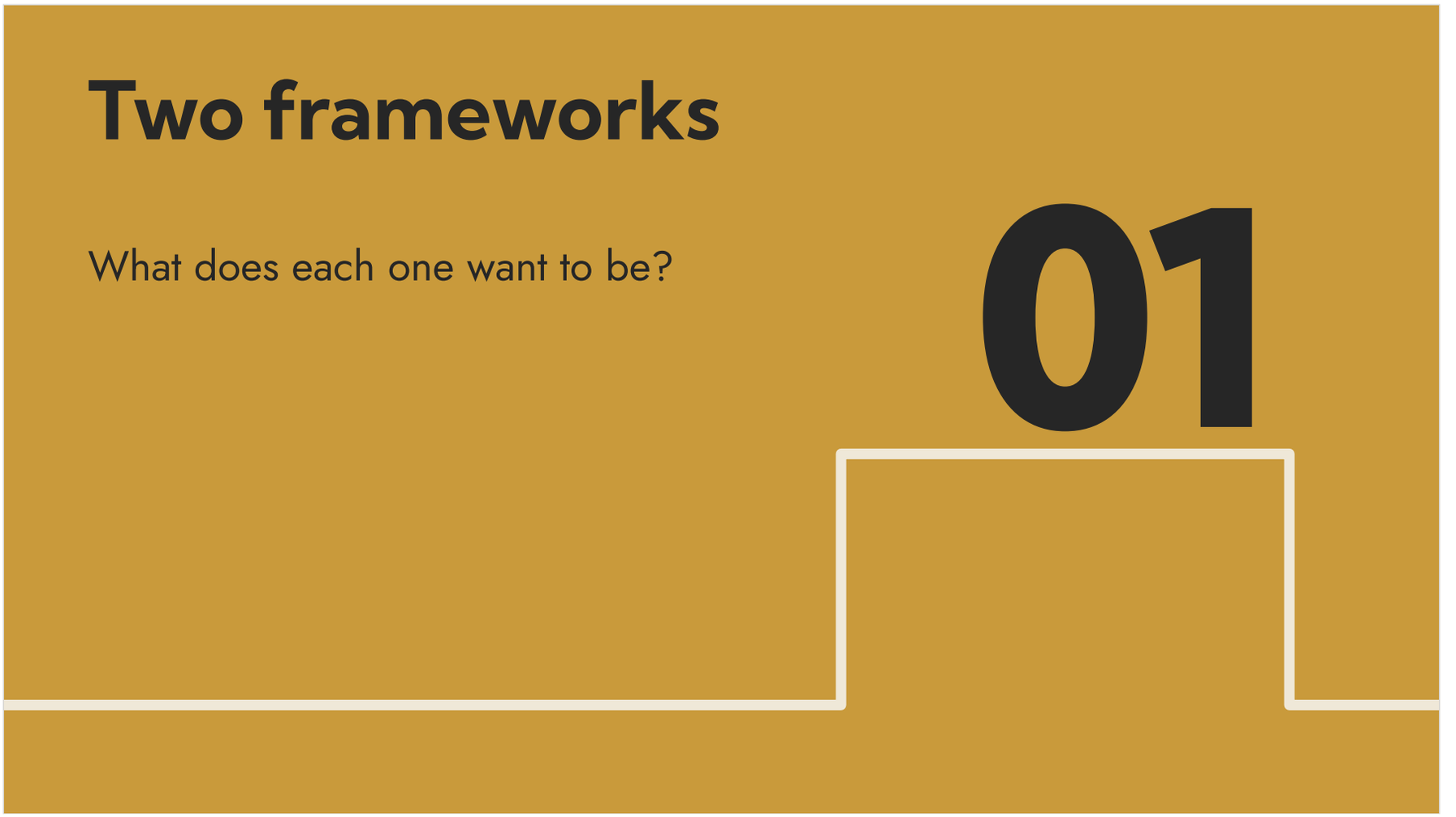
- Three answers, three parts.
- 1 someone else's model: Candle and the Hub.
- 2 one model in both: sin(x).
- 3 without std: Burn on an emulated microcontroller.

My question for this talk: what does it take to do machine learning in Rust?

I'll answer it three times. First, use someone else's model: an image classifier, a sentence model and a small chat model, with Candle and the Hugging Face Hub. Second, train one model in both frameworks from the same start, and check whether they agree. Third, leave the standard library behind: Burn on an emulated microcontroller, from a sine wave to a language model.

Before that, who these two are, and what a tensor looks like in each.

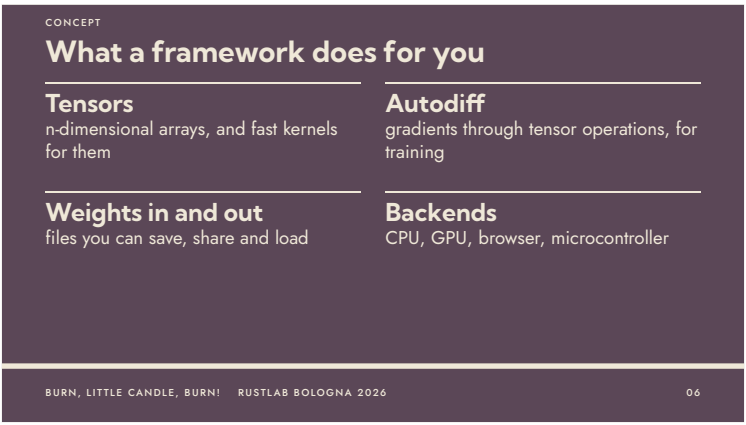
01 Two frameworks



- Section 1. Keep moving.

First, who are these two?

What a framework does for you



- **Four jobs. Point at each.**
- **Tensors, autodiff, weights, backends.**
- **Both do all four, with different priorities.**

Strip any ML framework down and it does four jobs.

Tensors: n-dimensional arrays, plus fast kernels like matrix multiplication.

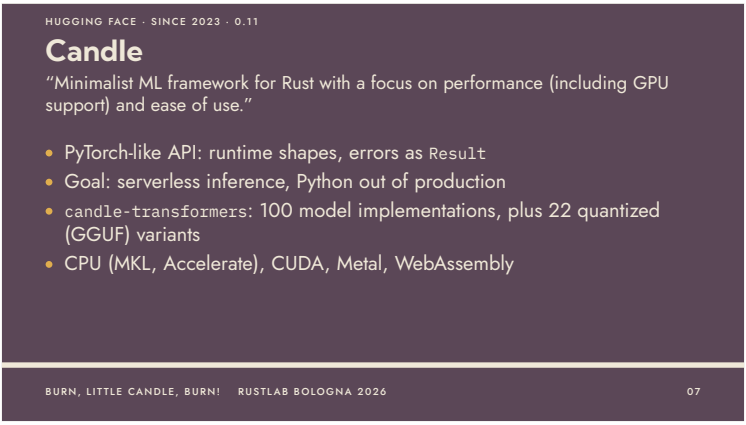
Autodiff: you write the forward computation, the framework works out the gradients. That is what makes training possible.

Weights in and out: a trained model is mostly a file full of numbers. You need to save it, load it, and share it with other tools.

Backends: the same model should run on a CPU, a GPU, in a browser, maybe on a microcontroller.

Candle and Burn both do all four. Where they differ is which job they consider the main one.

Candle



- **Hugging Face, first release August 2023, now 0.11.**
- **Inference first: serverless, no Python in production.**
- **100 models in candle-transformers (+22 quantized).**

Candle comes from Hugging Face. The first release on crates.io was in August 2023; the current one is 0.11.

Their own description is "minimalist ML framework with a focus on performance and ease of use". The README states the goals plainly: make serverless inference possible, small binaries that start fast, and remove Python from production workloads.

The API feels like PyTorch. Shapes are runtime values, and operations that can fail return a Result.

The big asset is candle-transformers: in 0.11 I counted 100 model implementations, plus 22 quantized variants that load llama.cpp GGUF files. Llama, Mistral, Qwen, Gemma, Whisper, Stable Diffusion, and many more.

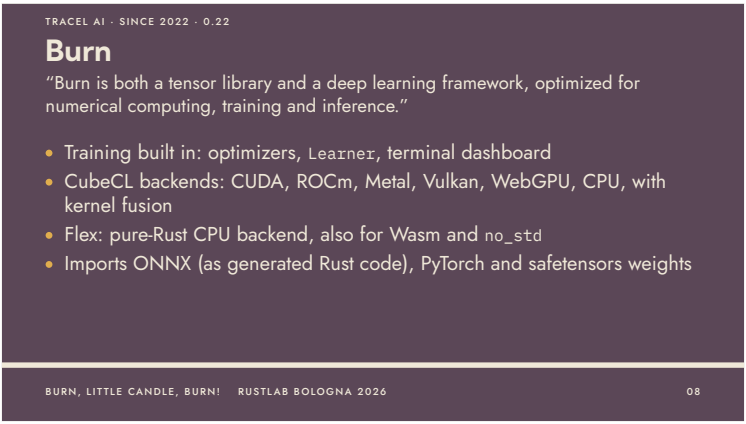
Backends: CPU, with MKL or Apple Accelerate as options; CUDA; Metal; and WebAssembly.

> **Q: Can Candle train?**
> A: Yes. It has autodiff (backward on any tensor) and optimizers like SGD and AdamW in candle-nn. We will train with it in a minute. It has no training framework around that, no learner or dashboard.

> **Q: Does Candle do ONNX?**
> A: There is a candle-onnx crate for evaluating ONNX models. I did not test it for this talk.

> **Q: Is Candle pure Rust?**
> A: Its kernels are, on the CPU. But candle-core 0.11 depends on Hugging Face's tokenizers crate with the onig feature on every non-wasm target, and that builds the Oniguruma C library. So you need a C compiler. Burn's Flex backend is pure Rust.

Burn



- **Tracel AI, first release July 2022, now 0.22.**
- **Full framework: training built in.**
- **CubeCL GPU backends; Flex for CPU, Wasm, no_std.**

Burn is older: the first release was in July 2022. It is developed by Tracel AI, and the current version is 0.22.

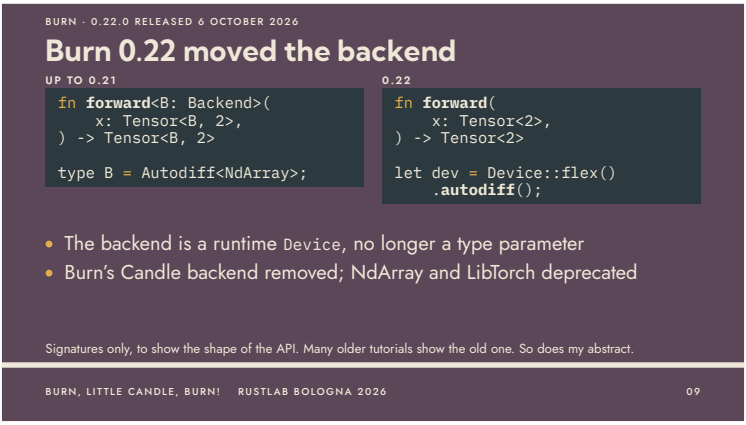
Burn wants to be a complete deep learning framework, for training and for inference. Training is built in: optimizers, a Learner that runs the loop for you, checkpoints, and a terminal dashboard written with Ratatui.

The GPU backends sit on CubeCL, Tracel's compute language: CUDA, ROCm, Metal, Vulkan and WebGPU, with automatic kernel fusion. For the CPU there are two: a CubeCL CPU backend, and Flex, a pure-Rust backend that also runs in WebAssembly and without the standard library.

For existing models, Burn imports ONNX by generating Rust code from it, and it loads PyTorch and safetensors weights.

> **Q: Is Burn stable?**
> A: The README says it is in active development and there will be breaking changes. The next slide shows a big one from three weeks ago.

Burn 0.22 moved the backend



- Three weeks ago: 0.22, 6 October.
- Left: backend as type parameter B. Right: runtime Device.
- Candle backend removed. Rank stays in the type.

If you looked at Burn before October, you saw code like the left side. Every model and every tensor carried a type parameter B for the backend, and you picked the backend with a type alias at the top of your program.

Burn 0.22 was released on 6 October, three weeks ago, and it removed that parameter. A tensor is now `Tensor<2>`, a model is just `Model`, and the backend is a runtime value: you create a Device, and you ask for autodiff on that device.

Two more changes from the migration guide: the Candle backend is gone, and the NdArray and LibTorch backends are deprecated. Yes, Burn could run on top of Candle until 0.21.

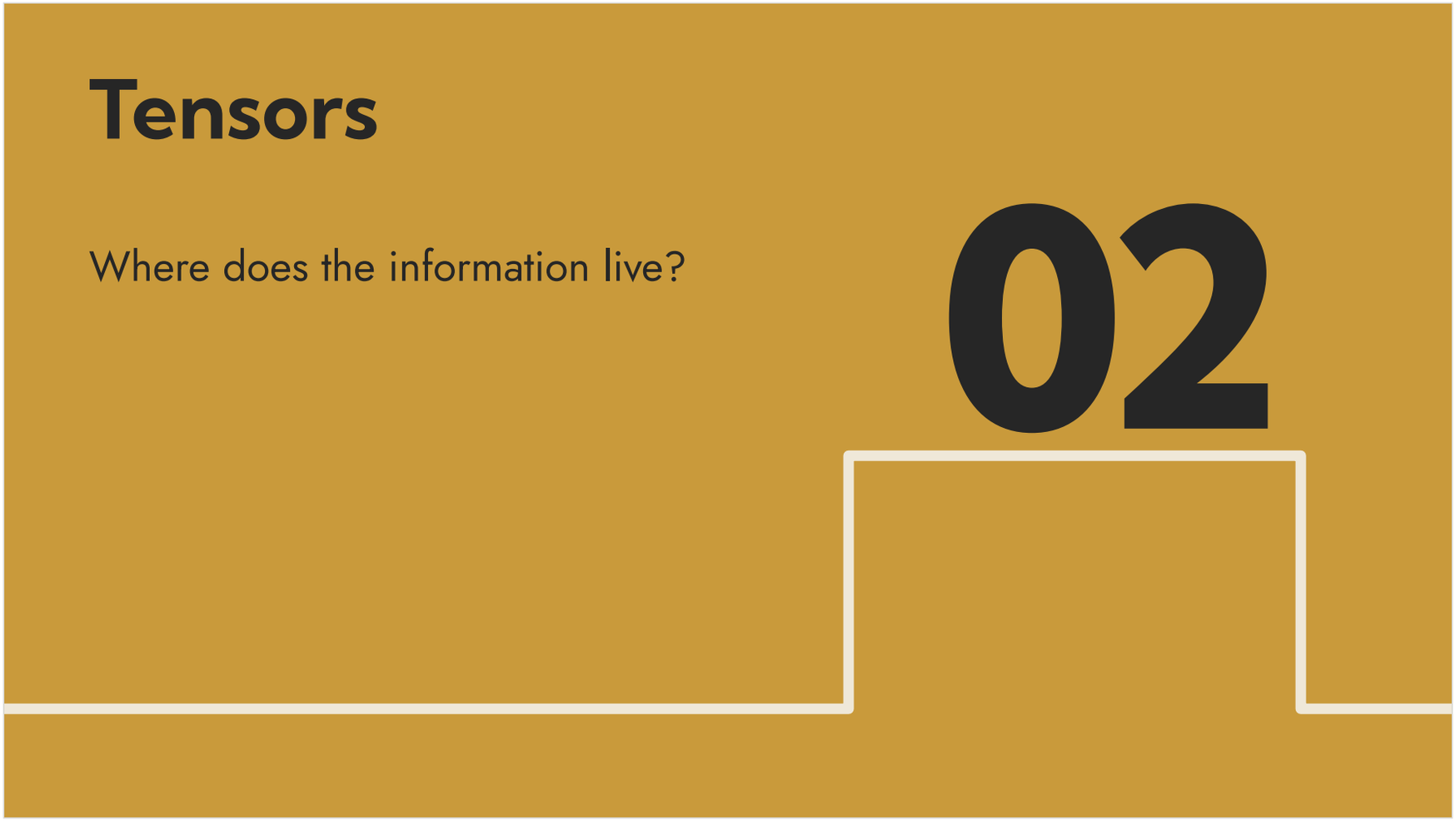
What did not change: the rank is still part of the type. That will matter in a minute.

And full disclosure: my abstract for this talk still mentions Burn's Backend trait. It was correct when I wrote it.

> **Q: Why did they do this?**
> A: The migration guide describes the new layering (tensor, bridge, dispatch, backend) but does not argue the motivation in detail. The visible effect is that ordinary model code no longer needs generics, and several backends can live in one program, chosen at run time.

> **Q: Will my old checkpoints load?**
> A: Not directly. 0.22 uses the burnpack format; the guide describes exporting old records to safetensors with burn-store in an older Burn and importing them.

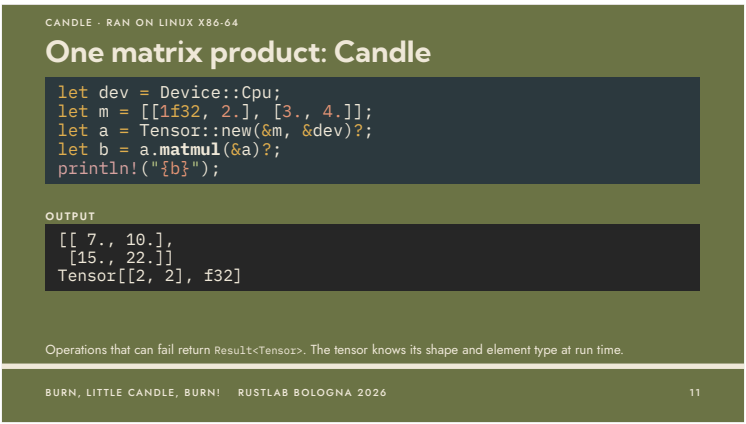
02 Tensors



- Section 2: where does the information live?

Let's look at the most basic thing, a tensor, and ask where each framework keeps the information about it.

One matrix product: Candle



- Point at Device::Cpu and every ?.
- Shape and dtype: runtime facts. Output shows them.

Here is one matrix product in Candle. You create a device, here the CPU. You create a tensor from a nested array. You multiply it with itself.

Notice the question marks: creating the tensor and the product can both fail, so both return a Result. And notice the borrow: matmul takes a reference to the other tensor.

The output prints the values, and below them what Candle considers the important metadata: the shape, two by two, and the element type, f32. Both are runtime values.

One matrix product: Burn



- No ?. Tensor::<2>: rank in the type.
- matmul takes self: clone first. Clone = refcount.
- Output shows device and kind.

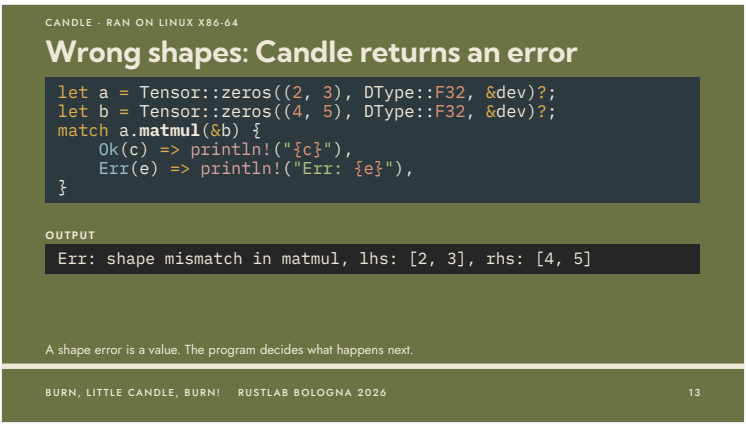
The same thing in Burn. No question marks: Burn operations do not return Results.

The `2` in `Tensor::<2>` is the rank, and it is part of the type.

And matmul takes `self`, by value. To use `a` twice, I clone it first. That looks expensive, but the Burn book explains that a clone only increments a reference count; the data is not copied. The ownership is the point: when Burn holds the only reference to a buffer, it can reuse it in place. Rust's ownership rules double as a memory optimizer.

The output also shows what Burn tracks: the device, Flex on the CPU, the kind, Float, and the dtype, f32.

Wrong shapes: Candle returns an error



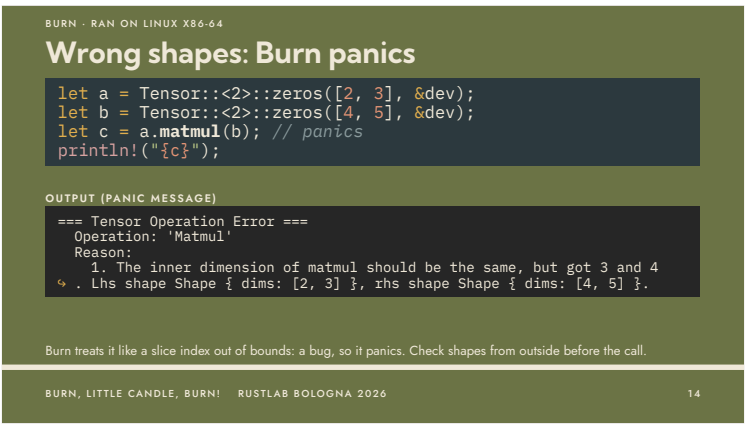
- [2,3] times [4,5]: impossible.
- Candle: Err value. Program keeps running.

Now let's make a mistake on purpose. A two by three matrix times a four by five matrix cannot work: the inner dimensions do not match.

Candle returns an Err. You can match on it, log it, recover. The message is short and precise: shape mismatch in matmul, with both shapes.

One detail I noticed: if RUST_BACKTRACE is set, Candle's error also carries a captured backtrace. That is handy while developing.

Wrong shapes: Burn panics



- Same mistake: Burn panics, exit code 101.
- Like slice indexing: a bug, not an error.
- Message is detailed: operation, reason, shapes.
- Shapes from outside: check before the call.

Same mistake in Burn. The program panics. The message is more verbose: a tensor operation error, the operation, the reason, and both shapes.

This is a design decision, not an omission. Burn treats a shape mismatch like indexing a slice out of bounds: a bug in your program, not a condition you are expected to handle. Candle treats it like PyTorch treats it, as an error value, roughly what an exception is in Python.

Neither is wrong. But it changes how you write code around them: in Candle you propagate errors; in Burn you check shapes that come from outside, from a file, a user or the network, before the call, and you test the rest.

Wrong rank: Burn does not compile



- Rank 2 times rank 1: compile error.
- expected 2, found 1.
- Fix: the vector as a [3, 1] matrix.
- Candle: compiles, then a runtime Err.

Now a different mistake: multiply a matrix by a vector, a rank 2 tensor by a rank 1 tensor. Burn's matmul wants both sides to have the same rank, and the rank is a const generic. So this does not compile. The compiler says: expected 2, found 1. As a three by one matrix, the vector works.

In Candle the same line compiles, and at run time returns the same shape mismatch error as before.

So Burn moved one piece of information, the rank, into the type system. The sizes of the dimensions stay runtime values in both frameworks. Candle's README is explicit about why it does not go further: it mentions dfdx, which puts full shapes into types, and says that approach can be daunting.

Where the information lives

CANDLE 0.11 · BURN 0.22

Where the information lives

	Candle	Burn
rank	value	type: `Tensor<2>`
shape	value	value
element type	value: `DType::F32`	kind in the type (`Int`), width a value
device	value: `Device::Cpu`	value: `Device::flex()`
shape error	`Err(…)`	panic
operations	borrow: `&self`	move: `self`, cheap clone

BURN, LITTLE CANDLE, BURN! RUSTLAB BOLOGNA 2026 16

- **Read the table row by row, briefly.**
- **Burn: rank and kind in types. Candle: everything a value.**
- **Since 0.22 both pick the device at run time.**

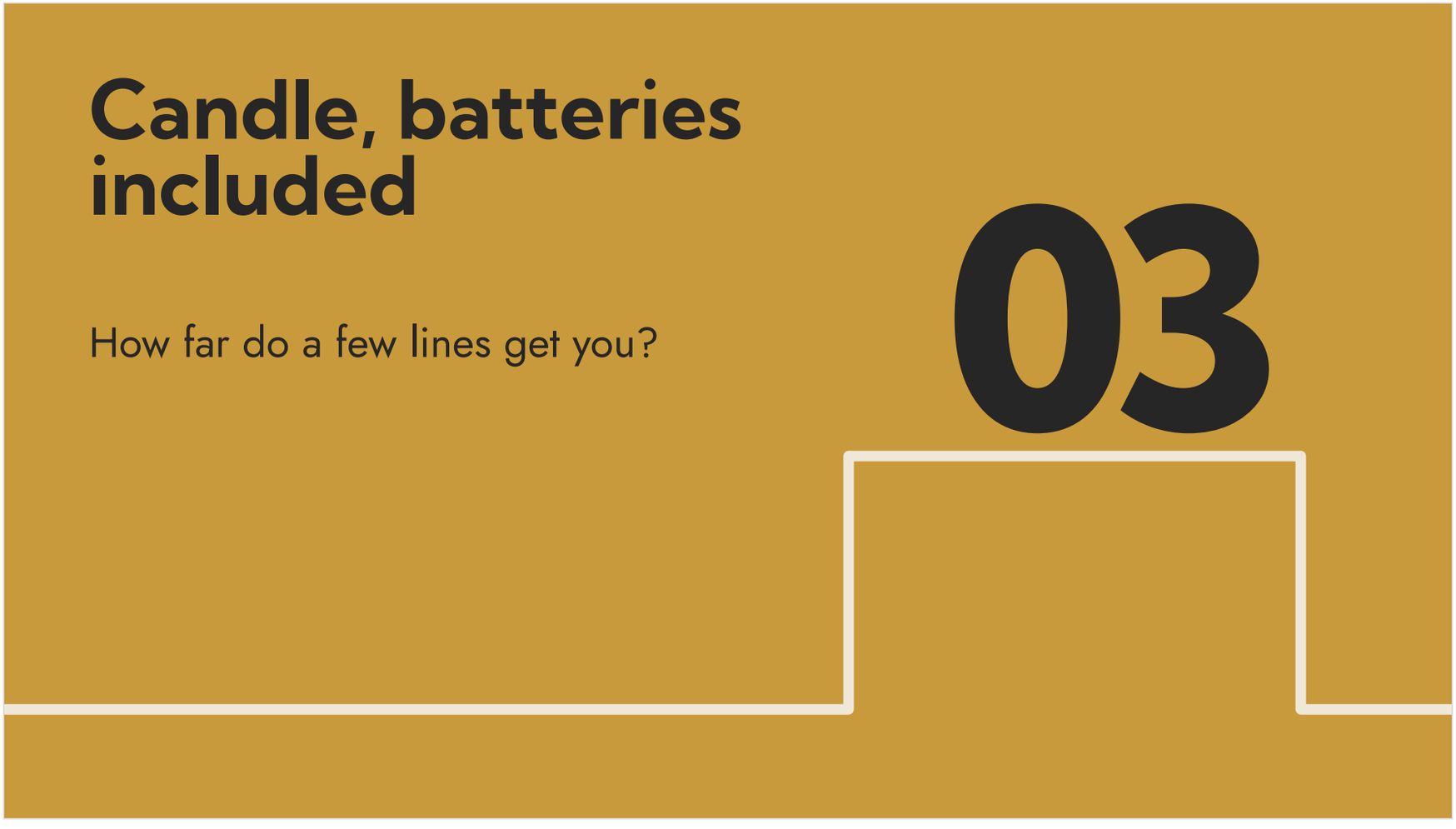
Here is the summary of the last slides.

The rank: a value in Candle, part of the type in Burn. The shape: a value in both. The element type: a value in Candle; in Burn the kind, float or int or bool, is in the type, while the width, f32 or f16, is a value. The device: a value in both, and in Burn only since 0.22.

Shape errors: an Err in Candle, a panic in Burn. And operations borrow in Candle and take ownership in Burn.

If you remember one row, remember this one: in Candle everything about a tensor is a runtime value, in Burn the rank and kind are compile-time facts.

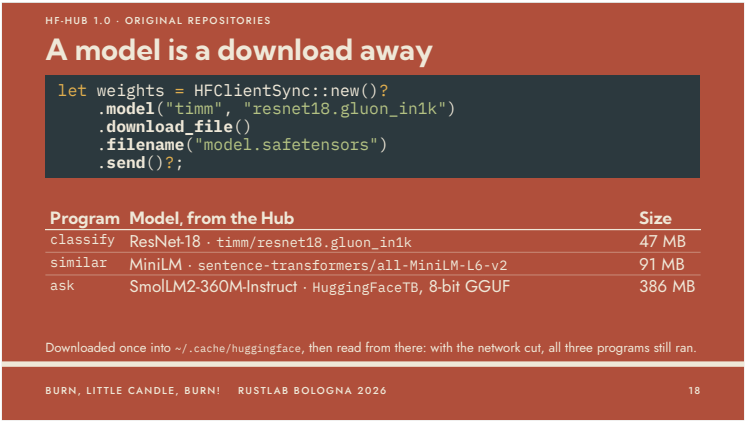
03 Candle, batteries included



- **Part one: someone else's model.**

Part one: someone else's model. This is Candle's home ground.

A model is a download away



- **hf-hub: name the repo, name the file, send.**
- **Three models, original publishers.**
- **Cached: offline after the first run.**

Candle comes from Hugging Face, and it shows. Getting a model is four lines with the hf-hub crate: name the repository, name the file, send. The file lands in the Hugging Face cache in your home directory, and later calls read it from there.

These are the three models of this part: ResNet-18 for images, MiniLM for sentences, and SmolLM2, a chat model with 360 million parameters, as an 8-bit GGUF file. All three come from the original publishers' repositories.

After the first run I cut the network, and all three programs still worked. On a conference Wi-Fi that matters.

> **Q: Why 8-bit and not 4-bit?**
> A: The publisher, HuggingFaceTB, offers only an 8-bit GGUF. 4-bit files exist from third parties; I used the original. Candle reads both.

> **Q: Where does the cache live?**
> A: ~/.cache/huggingface, unless HF_HOME says otherwise. Copy it to another machine and the programs run there offline.

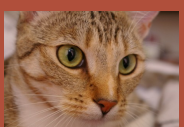
An image: ResNet-18

CANDLE 0.11 · CPU · RAN ON LINUX X86-64

An image: ResNet-18

```
let bytes = fs::read(weights)?;
let vb = VarBuilder::from_buffered_safetensors(bytes, DType::F32, &device)?;
let model = resnet::resnet18(1000, vb)?;

let logits = model.forward(&load_image(&path, &device)?)?;
let probs = softmax_last_dim(&logits)?.squeeze(0)?.to_vec1::<f32>()?;
```



CARGO CLASSIFY 2-CLASSIFY/IMAGES/CAT.JPG

```
81.7 %  Egyptian cat
  9.7 %  tiger cat
  8.3 %  tabby, tabby cat
  0.3 %  lynx, catamount
  0.0 %  washer, automatic washer, washing machine
```

The architecture comes from `candle-transformers`, the weights from the Hub. Scaling the image like the training images is your job.

- Bytes into a VarBuilder: tensors by name.
- Architecture from candle-transformers.
- Forward, softmax: 81.7 % Egyptian cat.
- Scaling the image is your job.

The image classifier. The bytes from the Hub go into a VarBuilder, which hands out tensors by name. The architecture comes from candle-transformers: resnet18, with 1 000 classes. One forward pass, and softmax turns the scores into probabilities.

The cat is from scikit-image's sample data: 81.7 percent Egyptian cat, then tiger cat and tabby. The coffee picture in the repository gives 95.5 percent espresso.

What you don't see is the function that scales the image to 224 by 224 and normalizes it with ImageNet's mean and standard deviation. That part is yours. Get it wrong and the model still answers, just worse.

A sentence: MiniLM



- Model: one vector per token, 384 numbers.
- Mask twice: in the model and in the average.
- Count the padding: correct Rust, wrong embeddings.
- Divide by the length: unit vectors.
- Dot product = cosine similarity.

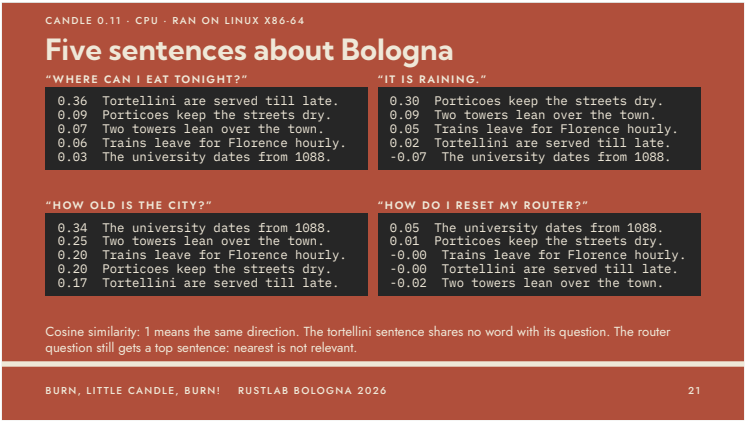
Sentence similarity with MiniLM, the model from my Hacker News project. The model gives one vector of 384 numbers per token. For a sentence you want one vector.

So you average over the tokens, but only over the real ones: the batch is padded to the longest sentence, and the attention mask says which positions are padding. The mask goes in twice: into the model, so attention skips the padding, and into the average. Then divide each vector by its length. Now every vector has length one, and a plain matrix product gives the cosine similarity of the question with every sentence.

Get the padding wrong and nothing complains: it compiles, it runs, and every vector is a little off. Correct Rust, wrong embeddings. That is why I checked these numbers against the official ONNX version of the model, run with ONNX Runtime: they agree to better than a millionth.

> **Q: Did you check that against Python?**
> A: Against the official ONNX export of the same model revision, run with ONNX Runtime, one sentence at a time and without padding. The largest difference in any of the 20 scores on the next slide is 0.0000003. The check is in `tools/minilm-reference`.

Five sentences about Bologna



- Eat: tortellini, no shared word.
- Rain: porticoes.
- How old: the university. Related, not the answer.
- Router: still a winner, at 0.05. Nearest is not relevant.

Five sentences about Bologna, four questions. "Where can I eat tonight?": the tortellini sentence wins, although the two share no word. "It is raining": the porticoes, which keep the streets dry. "How old is the city?": the university, from 1088. Related, not the answer: the university is not the city, but it is the only sentence with a date.

The scores are small, 0.3 or so, but the order is right, and the order is what you use.

And one question none of the sentences can answer: "How do I reset my router?" It still gets a winner: the university, at 0.05. There is always a nearest sentence. A low score is a hint, but where to draw the line depends on your data, so you have to measure it.

A chat: SmolLM2, one token at a time

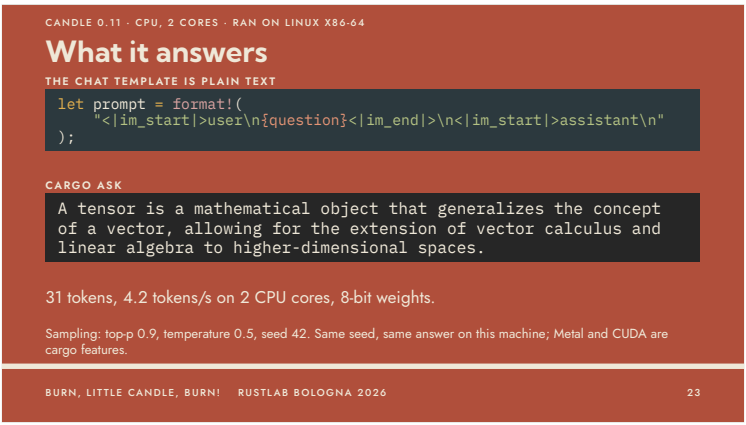


- First pass: the whole prompt.
- Then one token per step; pos = what the model has seen.
- KV cache inside ModelWeights.
- Sample, stop at the end marker, stream.

The chat model. This loop is all of text generation. The first time round, the input is the whole prompt. After that, only the newest token: pos tells the model how many tokens it has already seen, and the KV cache inside the model keeps their keys and values, so nothing is computed twice.

The model returns a score for every possible next token, the sampler picks one, and if it is the end marker we stop. Otherwise append it and print it as soon as the tokenizer can turn it into text: one token is not always a whole character.

What it answers



- Chat template: plain text from fine-tuning.
- One sentence, as asked.
- Speed: CPU, 2 cores; Metal is a feature.

The chat template is plain text, with markers the model saw during fine-tuning. Get it wrong and the model still talks, but it no longer answers.

The answer: one sentence, as asked. The speed on the slide is two CPU cores of a cloud VM. On a Mac, Metal is one cargo feature away; I did not measure that.

> **Q: How fast is it on a Mac?**
> A: Not measured for this talk. Build with –features metal; the README has the command.

What Candle gives you, what you write

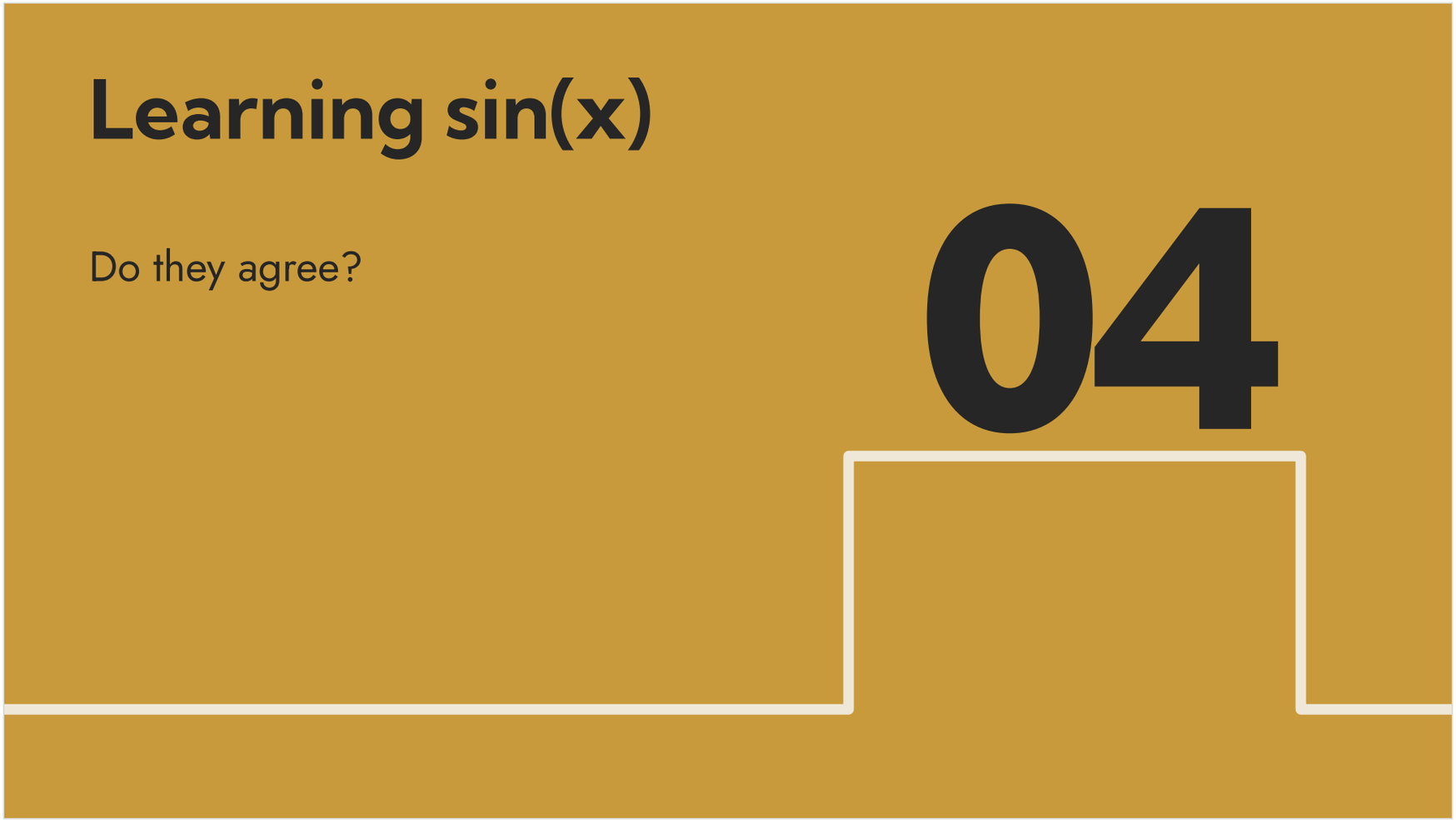


- **Left: Candle and friends.**
- **Right: your code, around the model.**
- **The right column is where bugs hide.**

The pattern behind all three programs. Candle and its neighbours give you the model code, the weights, the tokenizer, the GGUF reader and the sampler. You write the part around the model: scaling the image, averaging the token vectors, the chat template, the generation loop.

That right column is where bugs hide. A wrong preprocessing step does not crash; it makes the model quietly worse.

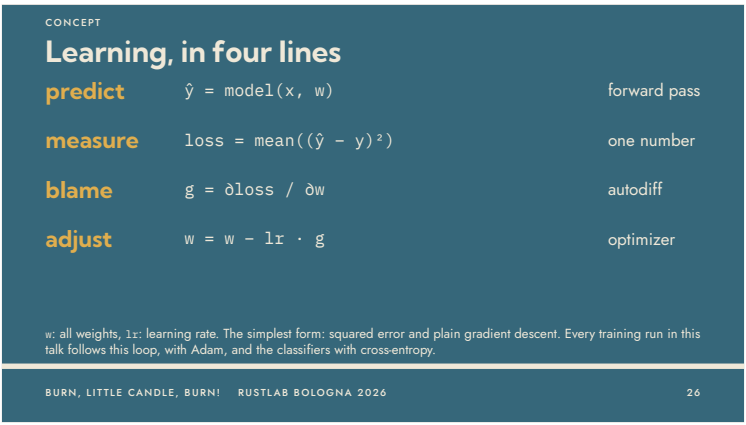
04 Learning sin(x)



- **Part two. Question: do they agree?**

Part two. If I give both frameworks exactly the same problem, do they agree?

Learning, in four lines



- **The only ML theory today. Four steps.**
- **predict, measure, blame, adjust. Repeat.**
- **Simplest form. Our runs: Adam; classifiers: cross-entropy.**

This is all the theory we need today.

A model is a function with weights, w. You feed it an input x and it predicts ŷ.

You measure how wrong the prediction is with a loss, here the mean squared error: one number.

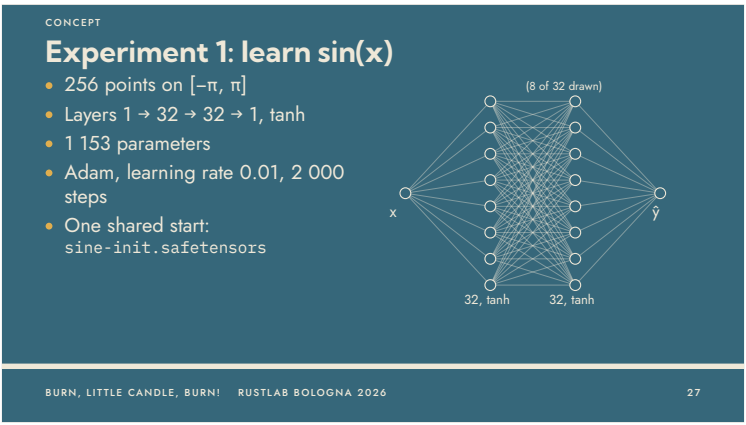
Then you blame the weights: autodiff computes the gradient, how much the loss changes when each weight changes.

Then you adjust: move every weight a small step against its gradient. The size of the step is the learning rate. The optimizer we use, Adam, is a smarter version of this step: it keeps running averages per weight.

Repeat until the loss stops falling. This is the simplest form. Every training run in this talk follows this loop, with Adam, and the two classifiers with cross-entropy as the loss.

> **Q: And the pretrained models?**
> A: Others trained them with the same loop, at a far larger scale. We only run them.

Experiment 1: learn sin(x)



- **256 points, 1-32-32-1, tanh, 1 153 weights.**
- **Key: one shared start file for both.**

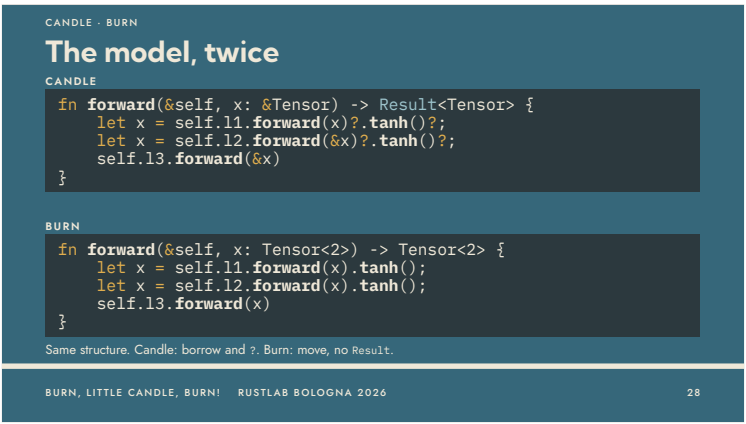
The task: learn the sine function between minus pi and pi from 256 sample points.

The network: one input, two hidden layers of 32 neurons with tanh, one output. 1 153 weights in total.

Training: Adam with learning rate 0.01, 2 000 steps, all 256 points at once.

And the important detail for a fair comparison: both frameworks start from the same initial weights, read from one safetensors file. I created that file once with Candle's initializer. Funny detail: Candle's CPU random number generator cannot be seeded; calling set_seed on the CPU device returns an error. So the file is also how this experiment stays reproducible.

The model, twice

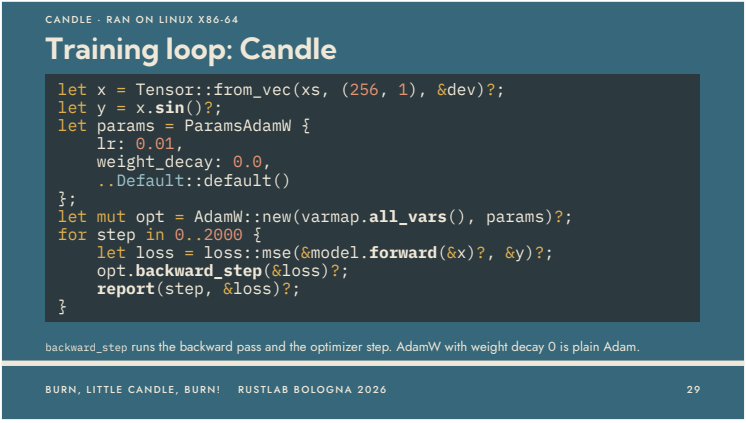


- Same structure, line by line.
- Candle: &x and ?. Burn: x by value.

Here is the forward pass in both frameworks. Line by line it is the same model: layer one, tanh, layer two, tanh, layer three.

The differences are exactly the ones from the tensor section. Candle takes a reference and returns a Result, so there is a question mark after every call. Burn takes the tensor by value and returns a tensor.

Training loop: Candle



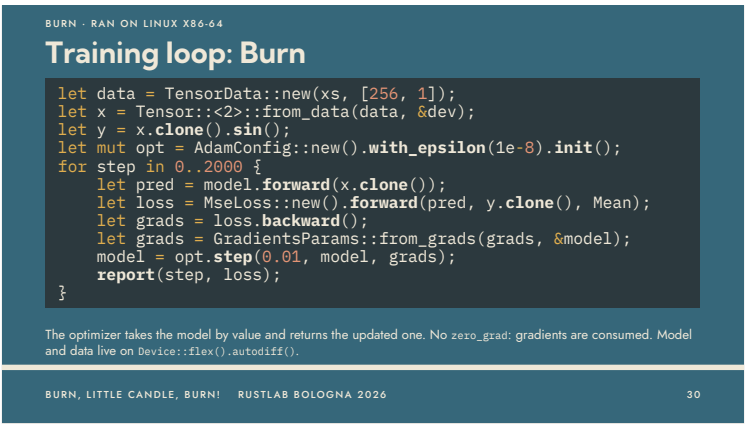
- AdamW with weight decay 0 = Adam.
- backward_step: backward plus update.

The training loop in Candle. Put the 256 x values into a tensor, compute y as the sine of x.

The optimizer is AdamW, with weight decay set to zero, which makes it plain Adam. Candle's default weight decay is 0.01, so this matters for the comparison.

The loop: compute the loss, then backward_step does the backward pass and the update in one call. report prints the loss every 250 steps.

Training loop: Burn



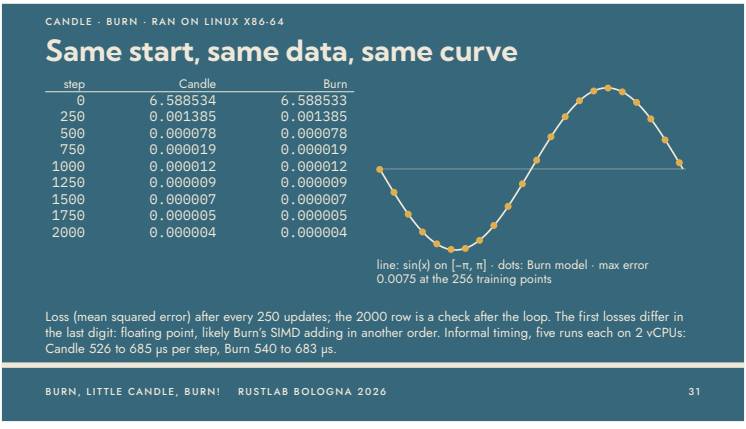
- Epsilon set to 1e-8 to match Candle (Burn default 1e-5).
- opt.step takes the model by value, returns new one.
- No zero_grad.

The same loop in Burn. One adjustment for fairness: Burn's Adam uses an epsilon of 1e-5 by default, Candle uses 1e-8, so I set Burn's to 1e-8.

Model and data live on Device::flex().autodiff(): the Flex CPU backend, with autodiff on top. Forward pass, loss, backward. The gradients come back as a value; GradientsParams maps them to the model's parameters. Then the optimizer takes the model by value and gives you back the updated model. Again ownership: the old model is consumed.

If you know PyTorch: there is no zero_grad. The gradients are a value you pass in, and they are consumed by the step.

Same start, same data, same curve



- Rows: loss after N updates; 2000 = check after the loop.
- Identical to six decimals from row 250 on.
- Row 0: last digit differs, likely SIMD summation order.
- Agreement checks the code; the plot checks the model.
- Plot: dots on the sine. Max error 0.0075, training points.
- Timing: five runs each, informal, not a benchmark.

And here is the answer: the loss after every 250 updates, Candle on the left, Burn on the right. The 2000 row is one more check after the loop.

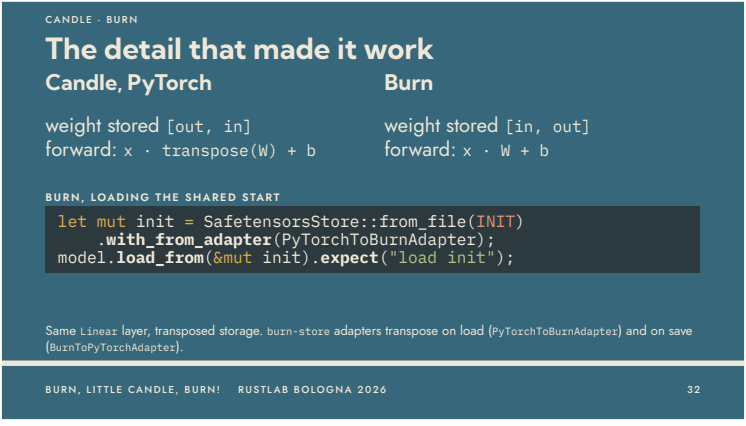
From row 250 on, the printed numbers are identical to six decimals. In the first row they differ in the last digit. That is floating point: most likely Burn's SIMD code adds in a different order, and a different order rounds differently.

So two frameworks, written by different teams with different designs, compute the same thing. That checks the code of both. Whether the model is any good is a different question, and that is the plot: the line is the real sine, the dots are the model. The worst error over the 256 training points is 0.0075. That checks the training, not new data.

The time per step is about the same for both, five runs each on two virtual CPUs. Informal, with a tiny model: it measures overhead more than arithmetic. No performance conclusions.

> **Q: So which one is faster?**
> A: I did not benchmark, and this model is far too small to say anything. Both use the gemm crate for CPU matrix products. For GPUs, it depends on the backend and the model; measure your own workload, with a warm-up, and wait for the GPU before you stop the clock (backup slide, Timing GPU code).

The detail that made it work



- Same Linear layer, transposed storage.
- PyTorchToBurnAdapter transposes on load.

One detail made the shared start file work, and it is a classic trap.

Candle follows PyTorch: a linear layer stores its weight as out by in, and the forward pass multiplies by the transpose. Burn stores in by out and multiplies directly. Same layer, same math, transposed storage.

So when Burn loads a PyTorch-style file it needs an adapter, PyTorchToBurnAdapter, which transposes the linear weights. And when Burn writes a file for others, BurnToPyTorchAdapter does the reverse. Forget the adapter and you get either a shape error or, worse for square matrices, silently wrong results.

05 Burn without std



- Part three: leave std behind.

Part three: leave the standard library behind.

Four steps, one no_std crate each



- No OS, no std; emulated M33, RAM cut to 520 KiB.
- Run, train, audio, pretrained LLM.
- One no_std library per step; shells around it.
- Candle needs std: Burn only.

The other end of the scale: no operating system, no standard library, and the RAM of a Pico 2, 520 KiB. This part is my burn-nostd repository.

Four steps, each a bit harder: run a model, train one on the device, handle audio, run someone else's pretrained model. Each step is one no_std library crate. The desktop programs, the browser modules and the firmware are thin shells around it. Even the desktop programs are no_std: libc stands in for std.

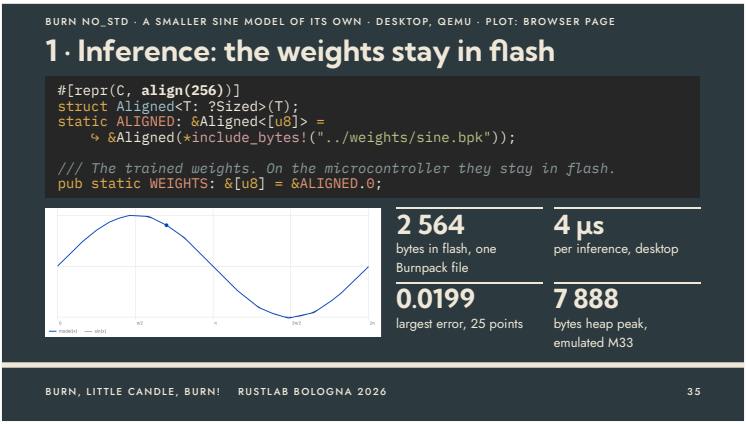
Everything in this part ran on the desktop, in the browser, and in QEMU on an emulated Cortex-M33 with the RAM limited to 520 KiB. The Pico 2 firmware builds; I have not run it on a board yet.

Candle is not in this part. It needs std: its build for the Pico's target stops at getrandom.

> **Q: Is there firmware for the real board?**
> A: Yes, the pico2 folder builds all four steps with Embassy. I have not run it on hardware yet. QEMU runs the same code paths with the same memory limit.

> **Q: Why the Flex backend?**
> A: It is Burn 0.22's portable CPU backend, pure Rust, and the one that builds without std.

1 · Inference: the weights stay in flash



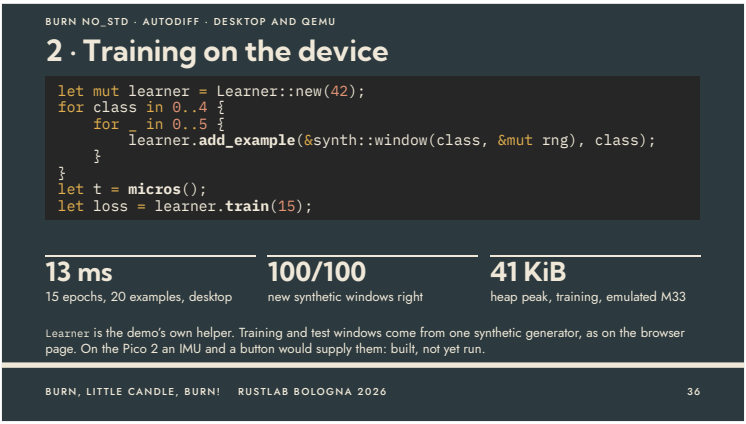
- A smaller sine model of its own: 321 parameters, ReLU, 0 to 2π.
- Burnpack file, include_bytes!, from_static.
- Trap: Burnpack wants 256-byte alignment.
- 2 564-byte file; heap peak under 8 KiB on the M33.

Step one: inference. This is a smaller sine model of its own, not the one from part two: 321 parameters, two hidden layers of 16 with ReLU, for x from 0 to 2π. The weights are a Burnpack file, compiled into the binary with include_bytes, and loaded with BurnpackStore::from_static. On the microcontroller they never leave flash.

The one trap: Burnpack wants its data aligned to 256 bytes, and include_bytes guarantees one. Hence the little wrapper struct.

The file is 2 564 bytes; the 321 weights themselves, as f32, are 1 284 of them. A few microseconds per inference on the desktop, and the largest error at 25 test points is 0.02. On the emulated Cortex-M33 the heap peaks under 8 KiB. The training program, also without std, reproduces this file byte for byte.

2 · Training on the device



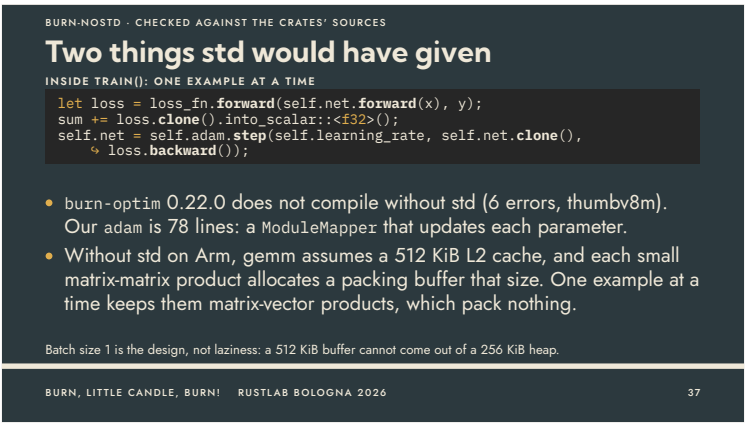
- Learner, the demo's own helper: five examples per gesture.
- train(15): autodiff without std.
- 100/100 new synthetic windows; heap peak 41 KiB on the M33.
- Pico 2: IMU and button, built, not yet run.

Step two: training on the device. Learner is a small helper of this demo, not Burn's Learner. It collects examples: five synthetic motion windows for each of four gestures, rest, shake, circle and punch. Then it trains for 15 epochs. Burn's autodiff works without std; the device is simply an autodiff device.

On the desktop that takes milliseconds, and the network gets all 100 new windows right. They come from the same synthetic generator as the training windows, so this shows that training works, not how well it would do on real motion. On the emulated Cortex-M33 the heap peaks at 41 KiB during training.

On the Pico 2 the examples would come from an IMU and a button. That firmware builds; I have not run it on a board yet.

Two things std would have given



- One example: forward, loss, backward, step.
- burn-optim: marked no_std, does not compile.
- gemm without std on Arm: 512 KiB buffer per small matrix-matrix product.
- Batch size 1: matrix-vector products, no packing. Heap: 256 KiB.

Inside train, one example at a time: forward, loss, backward, and an optimizer step. Two things std would have given me were missing.

First the optimizer. burn-optim 0.22.0 is marked no_std, but without std it does not compile for the Pico's target: six errors, a module that only exists with std and missing imports. So adam is a small crate of its own, 78 lines: a ModuleMapper that walks the parameters and updates each one.

Second, memory. Without std, gemm cannot ask the operating system for the cache sizes, so on Arm it assumes a 512 KiB L2 cache, and each small matrix-matrix product allocates a packing buffer of that size. Our heap is 256 KiB; the whole Pico 2 has 520 KiB. With one example at a time, every product is a matrix times a vector, and gemm computes those without packing.

> **Q: Did you report it?**
> A: Not yet. burn-optim without std and gemm's default are both worth an issue.

3 · Audio: teach it words



- Left to right; only the last box learns.
- Log-mel: own FFT, 16 bands.
- Words: louder than the measured noise floor.
- 10 slices × 16 bands, plus length: 161 inputs.
- 11/12 new synthetic words; heap peak 212 KiB on the M33.
- One synthetic espeak-ng voice: say so.

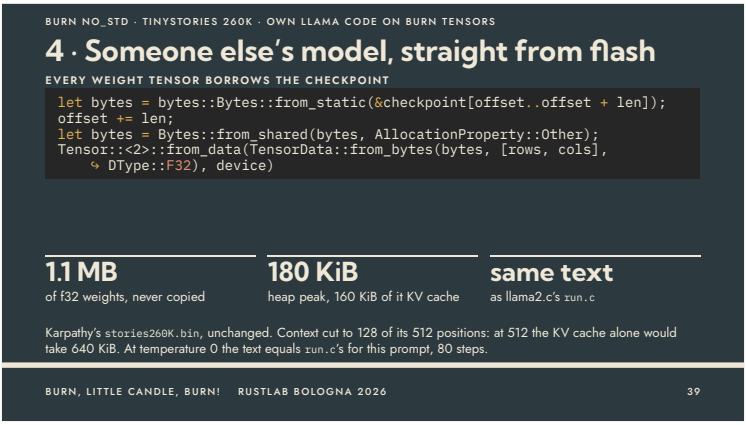
Step three: audio. Teach it words from five recordings each.

Left to right: 16 kilohertz audio; a log-mel spectrum, with an FFT written for this; a segmenter that finds words because they are louder than the noise floor it measured at the start; then features: ten time slices of 16 bands each, so a slow and a fast "yes" look alike, with the mean removed, so a loud and a quiet one do too, plus the length of the word. 161 numbers. Only the last box learns: a small network with 5 448 parameters.

Training on 20 recordings takes a fraction of a second on the desktop. Then it hears 12 new words and gets 11 right. On the emulated Cortex-M33 the heap peaks at 212 KiB. Honest footnote: all recordings are synthetic, made with one espeak-ng voice at random speed, pitch and loudness, the training words and the test words alike. Real voices and real microphones are not tested.

> **Q: Would it work with real voices?**
> A: The desktop program reads WAV files, so recordings of your own voice can go in. I have not measured that.

4 · Someone else's model, straight from flash



- Karpathy's TinyStories 260K, llama2.c file unchanged; own Llama code.
- from_static, from_shared, from_bytes: tensors borrow flash.
- 1.1 MB of f32 weights; heap peak 180 KiB, 160 of it KV cache.
- Context 128 of 512: at 512 the KV cache alone is 640 KiB.
- Same text as run.c at temperature 0.

Step four: someone else's pretrained model. Andrej Karpathy's TinyStories 260K, a tiny Llama trained on children's stories, in the file format of his llama2.c project, unchanged. The model code is my own: the Llama layers, written with Burn tensors.

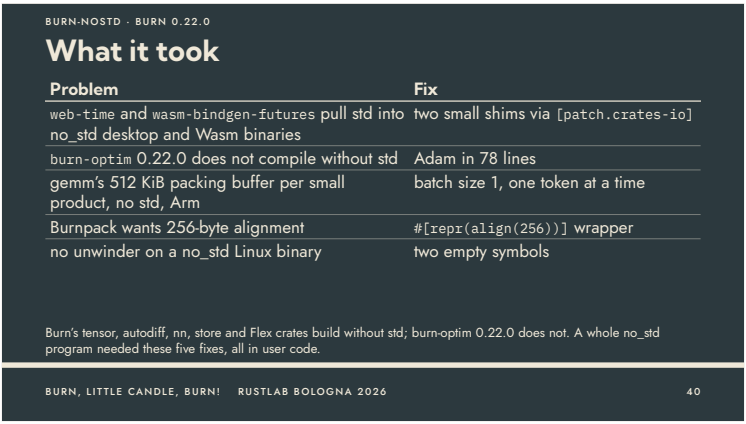
The trick is three calls. Bytes::from_static wraps a slice of the checkpoint in flash, from_shared hands it to Burn without copying, and TensorData::from_bytes makes it a tensor. Every weight matrix borrows the file. So 1.1 megabytes of f32 weights run in 520 KiB of RAM: the heap peaks at 180 KiB, and 160 of that is the KV cache.

That cache holds 128 positions. The model was trained with 512, and at 512 the KV cache alone would take 640 KiB, more than the whole chip. So the firmware cuts the context to 128.

At temperature zero the text is the same as from llama2.c's own C program, run.c: this prompt, 80 steps, compared as text. You saw that story on the second slide.

> **Q: Could it run a bigger model?**
> A: The desktop program takes other llama2.c files too; I only ran this one. The next size up, stories15M, has about 60 MB of f32 weights, far beyond a Pico 2, even with PSRAM.

What it took



- Core crates build without std: tensor, autodiff, nn, store, Flex.
- burn-optim 0.22.0 does not.
- A whole program: five fixes, all in user code. No fork.

What it took. Burn's core crates, tensor, autodiff, nn, store and the Flex backend, build without std; that part is real. burn-optim 0.22.0 does not. A complete no_std program ran into five things.

In the no_std desktop and WebAssembly binaries, web-time and wasm-bindgen-futures pull std back in: two small shims replace them. burn-optim does not compile without std. gemm's packing buffer, without std on Arm. Burnpack's alignment. And a no_std Linux binary still needs two unwinder symbols.

None of it needed a fork of Burn. The first three are worth upstream issues.

Try it now, on your phone



- QR: weinzierl.github.io/burn-nostd.
- Laptop: keywords page, sample voice, train, play.
- Fallback: cargo web offline, or the videos.

You can try all four steps now, on your phone. The same crates as the Pico 2 firmware, compiled to WebAssembly with zero imports, run in your browser, and nothing leaves the page.

On the laptop I'll show the keywords page: use the sample voice, train, play the sample sentence. You hear the words and see them recognized.

If the network or the audio fails: cargo web serves the four pages offline from the repository, and there are videos of all four.

06 Choosing



- **Section 6: which one, for what?**

So, which one should you use?

Which one?



- **Run a Hub model: Candle.**
- **Optional, your story: SmolLM2 on Burn 0.21, F32 against F16.**
- **Train, train on device, MCU: Burn.**
- **GPUs beyond CUDA/Metal, ONNX: Burn.**
- **Weights move between them: safetensors.**

My reading, as of today.

If you want to run a model that already exists, start with Candle: the model zoo, the tokenizers, GGUF, and the Hub at the door.

(Optional, your own story, in your words.) When I first tried SmolLM2 with Burn 0.21, it stopped at a type error: dtype mismatch, expected F32, got F16. Both frameworks support Llama-style models, but "supported" needs the whole sentence: this model, these operations, this dtype, this backend, this version.

If you want to train in Rust, Burn has autodiff, optimizers and a training loop. Training on the device, and microcontrollers in general: Burn, because its autodiff and its Flex backend work without std, and Candle needs std.

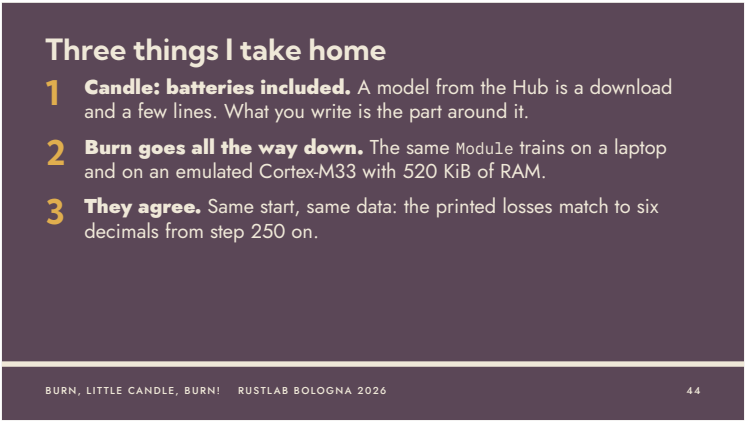
GPUs beyond CUDA and Metal: Burn, through CubeCL. I did not test GPUs for this talk. An ONNX model: Burn's importer writes Rust code from it; Candle's candle-onnx walks the graph at run time.

And you can use both: weights move between them through safetensors, as long as the tensor names match and you mind the layout of linear layers.

> **Q: Can I use both in one project?**
> A: Yes. Safetensors is the bridge, as in this talk. Since 0.22 Burn no longer runs on top of Candle, so the file is the interface.

> **Q: How do I benchmark this on a GPU?**
> A: Warm up first, because Burn's GPU backends autotune their kernels on the first calls. GPU work runs in the background, so wait for it before you stop the clock, with `device.synchronize()` in Candle or `device.sync()` in Burn, or use Burn's `device.profile`. Backup slide, Timing GPU code.

Three things I take home



- 1 Candle: batteries included.
- 2 Burn goes all the way down: laptop to emulated M33.
- 3 They agree: six decimals from step 250 on.

Three things I take home.

One: Candle is batteries included. A model from the Hub is a download and a few lines, and the code you write is the code around the model.

Two: Burn goes all the way down. The same Module trains on a laptop and on an emulated Cortex-M33 with 520 KiB of RAM.

Three: given the same start and the same data, they agree: the printed losses match to six decimals from step 250 on.

Try it



- Two QR codes. Pause five seconds.
- Weights download once; everything else is in the repos.

Everything is in two places. The talk repository has the slides, these notes, the Candle programs and both sine runs. burn-nostd has the four steps, and its page runs the demos.

(Pause for the QR codes.)

Grazie mille



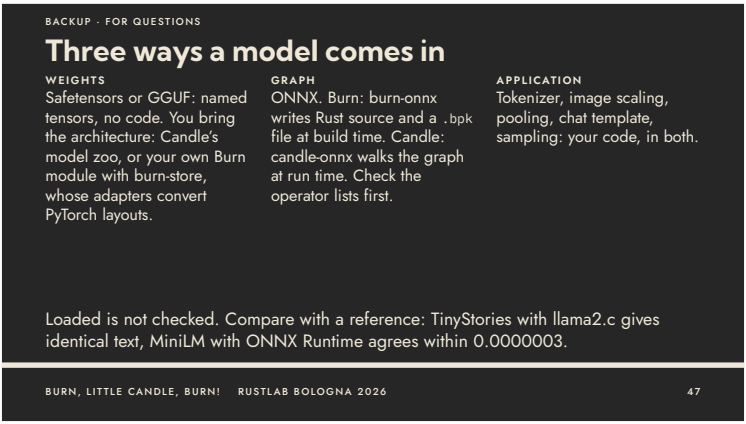
- Grazie mille. Questions.
- QR on this slide: the talk repository.
- Backup slides after this one: import, quantization, GPU timing.
- Backup notes: Burn 0.22, sine result, steps, step 2, step 4, choosing.

Grazie mille, thank you very much. I'm happy to take questions. The QR code on this slide is the talk repository, with the slides and these notes.

Backup answers are in the notes of the Burn 0.22 slide (why the change), the sine result (performance), Four steps (the real board), Training on the device (hardware status), Someone else's model (bigger models) and Which one (using both).

Three backup slides follow this one, for questions only: how a model comes in, quantization, and timing GPU code.

Backup: three ways a model comes in



- Weights: named tensors; you bring the architecture.
- Graph: burn-onnx writes Rust, candle-onnx walks the graph.
- Application code is yours in both.
- Loaded is not checked: compare with a reference.

For a question about bringing a model in.

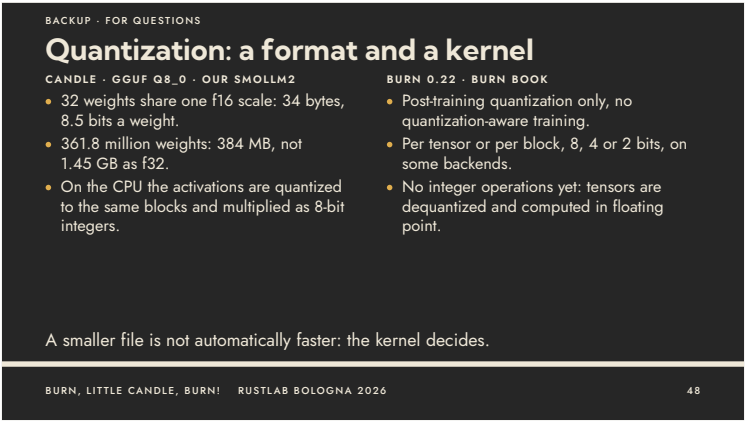
A weights file, safetensors or GGUF, is named tensors and no code. The architecture is code: in Candle it usually comes from the model zoo in candle-transformers; in Burn you write the module, and burn-store loads the weights, with adapters for the PyTorch layout of linear layers, as in the sine experiment.

An ONNX file also has the graph. Burn's importer, burn-onnx, turns it into Rust source and a `.bpk` weights file at build time. The Burn Book says that opset coverage does not mean every operator configuration works, so check the list. Candle's candle-onnx evaluates the graph at run time.

Tokenizer, image preprocessing, pooling, the chat template and sampling stay in your code with either.

And loading is where checking starts: TinyStories gave the same text as llama2.c, and the MiniLM numbers agree with ONNX Runtime to better than a millionth.

Backup: quantization



- **Q8_0: 32 weights, one f16 scale, 34 bytes.**
- **SmolLM2: 384 MB instead of 1.45 GB as f32.**
- **Burn 0.22: post-training only, dequantizes to compute.**

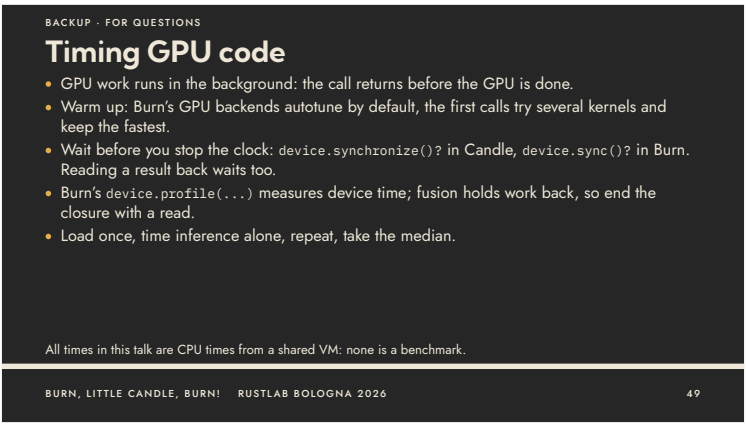
For a question about quantization.

The SmolLM2 file in the chat demo is GGUF with Q8_0: blocks of 32 weights that share one f16 scale, 34 bytes per block, 8.5 bits per weight. 361.8 million weights take 384 MB; as f32 the same weights would take 1.45 GB. On the CPU, Candle quantizes the activations into the same blocks and multiplies 8-bit integers.

Burn 0.22, according to its book: post-training quantization only, no quantization-aware training; per tensor or per block, 8, 4 or 2 bits, on some backends; and no integer operations yet, so tensors are dequantized and the arithmetic runs in floating point.

So a smaller file is not automatically a faster model: that depends on the kernel. I did not compare quantized and unquantized speed in either framework.

Backup: timing GPU code



- Calls return before the GPU is done.
- Warm up: autotune on the first calls.
- Synchronize before you stop the clock.
- All times in this talk: CPU, not benchmarks.

For a question about speed on a GPU.

GPU work is queued and runs in the background, so a timer around a call can stop before the GPU has done anything. Warm up first: Burn's GPU backends autotune by default, so the first calls try several kernels and keep the fastest. Then wait for the device before you stop the clock: `device.synchronize()` in Candle, `device.sync()` in Burn. Reading a result back to the CPU waits as well.

Burn also has `device.profile`, which measures device time. Its fusion backend holds the last operations of a closure back to batch them, so end the closure with a read.

Load the model once, time inference on its own, repeat, and take the median. All times in this talk are CPU times on a shared VM, and none of them is a benchmark.